

SUGGESTED SKILLS

1.A

Determine an appropriate program design to solve a problem or accomplish a task.

TOPIC 1.1

Introduction to Algorithms, Programming, and Compilers

Required Course Content

LEARNING OBJECTIVE

1.1.A

Represent patterns and algorithms found in everyday life using written language or diagrams.

1.1.B

Explain the code compilation and execution process.

1.1.C

Identify types of programming errors.

ESSENTIAL KNOWLEDGE

1.1.A.1

Algorithms define step-by-step processes to follow when completing a task or solving a problem. These algorithms can be represented using written language or diagrams.

1.1.A.2

Sequencing defines an order for when steps in a process are completed. Steps in a process are completed one at a time.

1.1.B.1

Code can be written in any text editor; however, an *integrated development environment (IDE)* is often used to write programs because it provides tools for a programmer to write, compile, and run code.

1.1.B.2

A *compiler* checks code for some errors. Errors detectable by the compiler need to be fixed before the program can be run.

1.1.C.1

A *syntax error* is a mistake in the program where the rules of the programming language are not followed. These errors are detected by the compiler.

continued on next page

LEARNING OBJECTIVE

1.1.C

Identify types of programming errors.

ESSENTIAL KNOWLEDGE

1.1.C.2

A *logic error* is a mistake in the algorithm or program that causes it to behave incorrectly or unexpectedly. These errors are detected by testing the program with specific data to see if it produces the expected outcome.

1.1.C.3

A *run-time error* is a mistake in the program that occurs during the execution of a program. Run-time errors typically cause the program to terminate abnormally.

1.1.C.4

An *exception* is a type of run-time error that occurs as a result of an unexpected error that was not detected by the compiler. It interrupts the normal flow of the program's execution.

SUGGESTED SKILLS

1.A

Determine an appropriate program design to solve a problem or accomplish a task.

2.A

Write program code to implement an algorithm.

TOPIC 1.2

Variables and Data Types

Required Course Content

LEARNING OBJECTIVE

1.2.A

Identify the most appropriate data type category for a particular specification.

1.2.B

Develop code to declare variables to store numbers and Boolean values.

ESSENTIAL KNOWLEDGE

1.2.A.1

A *data type* is a set of values and a corresponding set of operations on those values. Data types can be categorized as either primitive or reference.

1.2.A.2

The *primitive data types* used in this course define the set of values and corresponding operations on those values for numbers and Boolean values.

1.2.A.3

A *reference type* is used to define objects that are not primitive types.

1.2.B.1

The three primitive data types used in this course are `int`, `double`, and `boolean`. An `int` value is an integer. A `double` value is a real number. A `boolean` value is either `true` or `false`.

X EXCLUSION STATEMENT—*The other five primitive data types (long, short, byte, float, and char) are outside the scope of the AP Computer Science A course and exam.*

1.2.B.2

A *variable* is a storage location that holds a value, which can change while the program is running. Every variable has a name and an associated data type. A variable of a primitive type holds a primitive value from that type.

TOPIC 1.3

Expressions and Output

Required Course Content

LEARNING OBJECTIVE

1.3.A

Develop code to generate output and determine the result that would be displayed.

1.3.B

Develop code to utilize string literals and determine the result of using string literals.

1.3.C

Develop code for arithmetic expressions and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

1.3.A.1

`System.out.print` and `System.out.println` display information on the computer display. `System.out.println` moves the cursor to a new line after the information has been displayed, while `System.out.print` does not.

1.3.B.1

A *literal* is the code representation of a fixed value.

1.3.B.2

A *string literal* is a sequence of characters enclosed in double quotes.

1.3.B.3

Escape sequences are special sequences of characters that can be included in a string. They start with a `\` and have a special meaning in Java. Escape sequences used in this course include double quote `"`, backslash `\`, and newline `\n`.

1.3.C.1

Arithmetic expressions, which consist of numeric values, variables, and operators, include expressions of type `int` and `double`.

SUGGESTED SKILLS**2.A**

Write program code to implement an algorithm.

3.A

Determine the result or output based on statement execution order in an algorithm.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

continued on next page

LEARNING OBJECTIVE

1.3.C

Develop code for arithmetic expressions and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

1.3.C.2

The *arithmetic operators* consist of addition `+`, subtraction `-`, multiplication `*`, division `/`, and remainder `%`. An arithmetic operation that uses two `int` values will evaluate to an `int` value. An arithmetic operation that uses at least one `double` value will evaluate to a `double` value.

X EXCLUSION STATEMENT—*Expressions that result in special double values (e.g., infinities and NaN) are outside the scope of the AP Computer Science A course and exam.*

1.3.C.3

When dividing numeric values that are both `int` values, the result is only the integer portion of the quotient. When dividing numeric values that use at least one `double` value, the result is the quotient.

1.3.C.4

The remainder operator `%` is used to compute the remainder when one number `a` is divided by another number `b`.

X EXCLUSION STATEMENT—*The use of values less than 0 for `a` and the use of values less than or equal to 0 for `b` is outside the scope of the AP Computer Science A course and exam.*

1.3.C.5

Operators can be used to construct compound expressions. At compile time, numeric values are associated with operators according to operator precedence to determine how they are grouped. Parentheses can be used to modify operator precedence. Multiplication, division, and remainder have precedence over addition and subtraction. Operators with the same precedence are evaluated from left to right.

1.3.C.6

An attempt to divide an integer by the integer zero will result in an `ArithmeticException`.

X EXCLUSION STATEMENT—*The use of dividing by zero when one numeric value is a `double` is outside the scope of the AP Computer Science A course and exam.*

TOPIC 1.4

Assignment Statements and Input

SUGGESTED SKILLS

2.A

Write program code to implement an algorithm.

4.A

Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE

1.4.A

Develop code for assignment statements with expressions and determine the value that is stored in the variable as a result of these statements.

ESSENTIAL KNOWLEDGE

1.4.A.1

Every variable must be assigned a value before it can be used in an expression. That value must be from a compatible data type. A variable is initialized the first time it is assigned a value. Reference types can be assigned a new object or `null` if there is no object. The literal `null` is a special value used to indicate that a reference is not associated with any object.

1.4.A.2

The assignment operator `=` allows a program to initialize or change the value stored in a variable. The value of the expression on the right is stored in the variable on the left.

EXCLUSION STATEMENT—*The use of assignment operators inside expressions (e.g., `a = b = 4;` or `a[i] += 5`) is outside the scope of the AP Computer Science A course and exam.*

1.4.A.3

During execution, an expression is evaluated to produce a single value. The value of an expression has a type based on the evaluation of the expression.

1.4.B

Develop code to read input.

1.4.B.1

Input can come in a variety of forms, such as tactile, audio, visual, or text. The `Scanner` class is one way to obtain text input from the keyboard.

EXCLUSION STATEMENT—*Any specific form of input from the user is outside the scope of the AP Computer Science A course and exam.*

SUGGESTED SKILLS

2.A

Write program code to implement an algorithm.

4.A

Describe the behavior of a code segment or program.

TOPIC 1.5

Casting and Range of Variables

Required Course Content

LEARNING OBJECTIVE

1.5.A

Develop code to cast primitive values to different primitive types in arithmetic expressions and determine the value that is produced as a result.

1.5.B

Describe conditions when an integer expression evaluates to a value out of range.

ESSENTIAL KNOWLEDGE

1.5.A.1

The *casting operators* `(int)` and `(double)` can be used to convert from a `double` value to an `int` value (or vice versa).

1.5.A.2

Casting a `double` value to an `int` value causes the digits to the right of the decimal point to be truncated.

1.5.A.3

Some code causes `int` values to be automatically cast (widened) to `double` values.

1.5.A.4

Values of type `double` can be rounded to the nearest integer by `(int)(x + 0.5)` for non-negative numbers or `(int)(x - 0.5)` for negative numbers.

1.5.B.1

The constant `Integer.MAX_VALUE` holds the value of the largest possible `int` value. The constant `Integer.MIN_VALUE` holds the value of the smallest possible `int` value.

1.5.B.2

Integer values in Java are represented by values of type `int`, which are stored using a finite amount (4 bytes) of memory. Therefore, an `int` value must be in the range from `Integer.MIN_VALUE` to `Integer.MAX_VALUE` inclusive.

continued on next page

LEARNING OBJECTIVE

1.5.B

Describe conditions when an integer expression evaluates to a value out of range.

1.5.C

Describe conditions that limit accuracy of expressions.

ESSENTIAL KNOWLEDGE

1.5.B.3

If an expression would evaluate to an `int` value outside of the allowed range, an integer overflow occurs. The result is an `int` value in the allowed range but not necessarily the value expected.

1.5.C.1

Computers allot a specified amount of memory to store data based on the data type. If an expression would evaluate to a `double` that is more precise than can be stored in the allotted amount of memory, a round-off error occurs. The result will be rounded to the representable value. To avoid rounding errors that naturally occur, use `int` values.

EXCLUSION STATEMENT—Other special decimal data types that can be used to avoid rounding errors are outside the scope of the AP Computer Science A course and exam.

SUGGESTED SKILLS

3.A

Determine the result or output based on statement execution order in an algorithm.

TOPIC 1.6

Compound Assignment Operators

Required Course Content

LEARNING OBJECTIVE

1.6.A

Develop code for assignment statements with compound assignment operators and determine the value that is stored in the variable as a result.

ESSENTIAL KNOWLEDGE

1.6.A.1

Compound assignment operators `+=`, `-=`, `*=`, `/=`, and `%=` can be used in place of the assignment operator in numeric expressions. A compound assignment operator performs the indicated arithmetic operation between the value on the left and the value on the right and then assigns the result to the variable on the left.

1.6.A.2

The post-increment operator `++` and post-decrement operator `--` are used to add 1 or subtract 1 from the stored value of a numeric variable. The new value is assigned to the variable.

✖ EXCLUSION STATEMENT—The use of increment and decrement operators in prefix form (e.g., `++x`) is outside the scope of the AP Computer Science A course and exam. The use of increment and decrement operators inside other expressions (e.g., `arr[x++]`) is outside the scope of the AP Computer Science A course and exam.

TOPIC 1.7

Application Program Interface (API) and Libraries

SUGGESTED SKILLS

4.A

Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE**1.7.A**

Identify the attributes and behaviors of a class found in the libraries contained in an API.

ESSENTIAL KNOWLEDGE**1.7.A.1**

Libraries are collections of classes. An *application programming interface (API)* specification informs the programmer how to use those classes. Documentation found in API specifications and libraries is essential to understanding the attributes and behaviors of a class defined by the API. A *class* defines a specific reference type. Classes in the APIs and libraries are grouped into packages. Existing classes and class libraries can be utilized to create objects.

1.7.A.2

Attributes refer to the data related to the class and are stored in variables. *Behaviors* refer to what instances of the class can do (or what can be done with them) and are defined by methods.

SUGGESTED SKILLS

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

TOPIC 1.8

Documentation with Comments

Required Course Content

LEARNING OBJECTIVE

1.8.A

Describe the functionality and use of code through comments.

ESSENTIAL KNOWLEDGE

1.8.A.1

Comments are written for both the original programmer and other programmers to understand the code and its functionality, but are ignored by the compiler and are not executed when the program is run. Three types of comments in Java include `/* */`, which generates a block of comments; `//`, which generates a comment on one line; and `/** */`, which are Javadoc comments and are used to create API documentation.

1.8.A.2

A *precondition* is a condition that must be true just prior to the execution of a method in order for it to behave as expected. There is no expectation that the method will check to ensure preconditions are satisfied.

1.8.A.3

A *postcondition* is a condition that must always be true after the execution of a method. Postconditions describe the outcome of the execution in terms of what is being returned or the current value of the attributes of an object.

TOPIC 1.9

Method Signatures

SUGGESTED SKILLS

3.C

Determine the result or output based on code that contains procedural abstractions.

Required Course Content

LEARNING OBJECTIVE

1.9.A

Identify the correct method to call based on documentation and method signatures.

1.9.B

Describe how to call methods.

ESSENTIAL KNOWLEDGE

1.9.A.1

A *method* is a named block of code that only runs when it is called. A *block of code* is any section of code that is enclosed in braces. *Procedural abstraction* allows a programmer to use a method by knowing what the method does even if they do not know how the method was written.

1.9.A.2

A *parameter* is a variable declared in the header of a method or constructor and can be used inside the body of the method. This allows values or arguments to be passed and used by a method or constructor. A *method signature* for a method with parameters consists of the method name and the ordered list of parameter types. A method signature for a method without parameters consists of the method name and an empty parameter list.

1.9.B.1

A *void method* does not have a return value and is therefore not called as part of an expression.

1.9.B.2

A *non-void method* returns a value that is the same type as the return type in the header. To use the return value when calling a non-void method, it must be stored in a variable or used as part of an expression.

continued on next page

LEARNING OBJECTIVE

1.9.B

Describe how to call methods

ESSENTIAL KNOWLEDGE

1.9.B.3

An *argument* is a value that is passed into a method when the method is called. The arguments passed to a method must be compatible in number and order with the types identified in the parameter list of the method signature. When calling methods, arguments are passed using call by value. *Call by value* initializes the parameters with copies of the arguments.

1.9.B.4

Methods are said to be *overloaded* when there are multiple methods with the same name but different signatures.

1.9.B.5

A *method call* interrupts the sequential execution of statements, causing the program to first execute the statements in the method before continuing. Once the last statement in the method has been executed or a return statement is executed, the flow of control is returned to the point immediately following where the method was called.

TOPIC 1.10

Calling Class Methods

Required Course Content

LEARNING OBJECTIVE

1.10.A

Develop code to call class methods and determine the result of those calls.

ESSENTIAL KNOWLEDGE

1.10.A.1

Class methods are associated with the class, not instances of the class. Class methods include the keyword `static` in the header before the method name.

1.10.A.2

Class methods are typically called using the class name along with the dot operator. When the method call occurs in the defining class, the use of the class name is optional in the call.

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

3.C

Determine the result or output based on code that contains procedural abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

4.A

Describe the behavior of a code segment or program.

TOPIC 1.11

Math Class

Required Course Content

LEARNING OBJECTIVE

1.11.A

Develop code to write expressions that incorporate calls to built-in mathematical libraries and determine the value that is produced as a result.

ESSENTIAL KNOWLEDGE

1.11.A.1

The `Math` class is part of the `java.lang` package. Classes in the `java.lang` package are available by default.

1.11.A.2

The `Math` class contains only class methods. The following `Math` class methods—including what they do and when they are used—are part of the Java Quick Reference:

- `static int abs(int x)` returns the absolute value of an `int` value.
- `static double abs(double x)` returns the absolute value of a `double` value.
- `static double pow(double base, double exponent)` returns the value of the first parameter raised to the power of the second parameter.
- `static double sqrt(double x)` returns the nonnegative square root of a `double` value.
- `static double random()` returns a `double` value greater than or equal to `0.0` and less than `1.0`.

1.11.A.3

The values returned from `Math.random()` can be manipulated using arithmetic and casting operators to produce a random `int` or `double` in a defined range based on specified criteria. Each endpoint of the range can be *inclusive*, meaning the value is included, or *exclusive*, meaning the value is not included.

TOPIC 1.12

Objects: Instances of Classes

SUGGESTED SKILLS

4.A

Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE

1.12.A

Explain the relationship between a class and an object.

1.12.B

Develop code to declare variables to store reference types.

ESSENTIAL KNOWLEDGE

1.12.A.1

An *object* is a specific instance of a class with defined attributes. A *class* is the formal implementation, or blueprint, of the attributes and behaviors of an object.

1.12.A.2

A class hierarchy can be developed by putting common attributes and behaviors of related classes into a single class called a *superclass*. Classes that extend a superclass, called *subclasses*, can draw upon the existing attributes and behaviors of the superclass without replacing these in the code. This creates an *inheritance relationship* from the subclasses to the superclass.

EXCLUSION STATEMENT—*Designing and implementing inheritance relationships are outside the scope of the AP Computer Science A course and exam.*

1.12.A.3

All classes in Java are subclasses of the `Object` class.

1.12.B.1

A variable of a reference type holds an object reference, which can be thought of as the memory address of that object.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

TOPIC 1.13

Object Creation and Storage (Instantiation)

Required Course Content

LEARNING OBJECTIVE

1.13.A

Identify, using its signature, the correct constructor being called.

1.13.B

Develop code to declare variables of the correct types to hold object references.

1.13.C

Develop code to create an object by calling a constructor.

ESSENTIAL KNOWLEDGE

1.13.A.1

A class contains *constructors* that are called to create objects. They have the same name as the class.

1.13.A.2

A *constructor signature* consists of the constructor's name, which is the same as the class name, and the ordered list of parameter types. The parameter list, in the header of a constructor, lists the types of the values that are passed and their variable names.

1.13.A.3

Constructors are said to be overloaded when there are multiple constructors with different signatures.

1.13.B.1

A variable of a reference type holds an object reference or, if there is no object, `null`.

1.13.C.1

An object is typically created using the keyword `new` followed by a call to one of the class's constructors.

1.13.C.2

Parameters allow constructors to accept values to establish the initial values of the attributes of the object.

continued on next page

LEARNING OBJECTIVE

1.13.C

Develop code to create an object by calling a constructor.

ESSENTIAL KNOWLEDGE

1.13.C.3

A *constructor argument* is a value that is passed into a constructor when the constructor is called. The arguments passed to a constructor must be compatible in order and number with the types identified in the parameter list in the constructor signature. When calling constructors, arguments are passed using call by value. Call by value initializes the parameters with copies of the arguments.

1.13.C.4

A constructor call interrupts the sequential execution of statements, causing the program to first execute the statements in the constructor before continuing. Once the last statement in the constructor has been executed, the flow of control is returned to the point immediately following where the constructor was called.

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

3.C

Determine the result or output based on code that contains procedural abstractions.

TOPIC 1.14

Calling Instance Methods

Required Course Content

LEARNING OBJECTIVE**1.14.A**

Develop code to call instance methods and determine the result of these calls.

ESSENTIAL KNOWLEDGE**1.14.A.1**

Instance methods are called on objects of the class. The dot operator is used along with the object name to call instance methods.

1.14.A.2

A method call on a `null` reference will result in a `NullPointerException`.

TOPIC 1.15

String Manipulation

Required Course Content

LEARNING OBJECTIVE

1.15.A

Develop code to create string objects and determine the result of creating and combining strings.

ESSENTIAL KNOWLEDGE

1.15.A.1

A `String` object represents a sequence of characters and can be created by using a string literal or by calling the `String` class constructor.

1.15.A.2

The `String` class is part of the `java.lang` package. Classes in the `java.lang` package are available by default.

1.15.A.3

A `String` object is immutable, meaning once a `String` object is created, its attributes cannot be changed. Methods called on a `String` object do not change the content of the `String` object.

1.15.A.4

Two `String` objects can be concatenated together or combined using the `+` or `+=` operator, resulting in a new `String` object. A primitive value can be concatenated with a `String` object. This causes the implicit conversion of the primitive value to a `String` object.

continued on next page

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

3.C

Determine the result or output based on code that contains procedural abstractions.

4.A

Describe the behavior of a code segment or program.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

LEARNING OBJECTIVE

1.15.A

Develop code to create string objects and determine the result of creating and combining strings.

1.15.B

Develop code to call methods on string objects and determine the result of calling these methods.

ESSENTIAL KNOWLEDGE

1.15.A.5

A `String` object can be concatenated with any object, which implicitly calls the object's `toString` method (a behavior that is guaranteed to exist by the inheritance relationship every class has with the `Object` class). An object's `toString` method returns a string value representing the object. Subclasses of `Object` often override the `toString` method with class-specific implementation. *Method overriding* occurs when a public method in a subclass has the same method signature as a public method in the superclass, but the behavior of the method is specific to the subclass.

X EXCLUSION STATEMENT—*Overriding the `toString` method of a class is outside the scope of the AP Computer Science A course and exam.*

1.15.B.1

A `String` object has index values from 0 to one less than the length of the string. Attempting to access indices outside this range will result in an `IndexOutOfBoundsException`.

continued on next page

LEARNING OBJECTIVE

1.15.B

Develop code to call methods on string objects and determine the result of calling these methods.

ESSENTIAL KNOWLEDGE

1.15.B.2

The following `String` methods—including what they do and when they are used—are part of the Java Quick Reference:

- `int length()` returns the number of characters in a `String` object.
- `String substring(int from, int to)` returns the substring beginning at index `from` and ending at index `to - 1`.
- `String substring(int from)` returns `substring(from, length())`.
- `int indexOf(String str)` returns the index of the first occurrence of `str`; returns `-1` if not found.
- `boolean equals(Object other)` returns `true` if this corresponds to the same sequence of characters as `other`; returns `false` otherwise.
- `int compareTo(String other)` returns a value `< 0` if this is less than `other`; returns zero if this is equal to `other`; returns a value `> 0` if this is greater than `other`. Strings are ordered based upon the alphabet.

EXCLUSION STATEMENT—Using the `equals` method to compare one `String` object with an object of a type other than `String` is outside the scope of the AP Computer Science A course and exam.

1.15.B.3

A string identical to the single element substring at position `index` can be created by calling `substring(index, index + 1)`.